МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №4

з курсу: «Інтелектуальні методи діагностики за медичними зображеннями»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2026

Завдання 23 Інтеграція BatchNorm у VAE та аналіз стабільності навчання

Мета: стабілізувати оптимізацію.

Теорія: внутрішня нормалізація активацій.

Очікуваний результат: швидша збіжність.

Датасет: NIH

Теоретичні відомості

Варіаційний автоенкодер є різновидом нейронної мережі, яка використовується для навчання ефективного представлення даних у латентному просторі. На відміну від класичного автоенкодера, VAE не просто стискає інформацію, а формує ймовірнісну модель розподілу даних. Це досягається за рахунок того, що енкодер замість одного вектора повертає параметри розподілу, зокрема середнє значення та дисперсію, з яких генерується латентний вектор.

Основною ідеєю використання VAE для виявлення аномалій є навчання моделі виключно на нормальних даних. У процесі навчання модель запам’ятовує характерні особливості здорових зображень. При подачі на вхід аномальних даних, таких як патологічні знімки, модель не може коректно їх відновити, що призводить до збільшення помилки реконструкції.

Функція втрат у VAE складається з двох компонент. Перша складова відповідає за якість реконструкції зображення. Друга складова є регуляризаційною і забезпечує наближення латентного простору до нормального розподілу. Це дозволяє отримати більш структуроване представлення даних.

Batch Normalization є методом нормалізації активацій нейронної мережі під час навчання. Його використання дозволяє зменшити внутрішнє зміщення розподілу даних між шарами мережі. Це сприяє стабільнішому та швидшому навчанню, зменшує ризик вибуху або затухання градієнтів і покращує збіжність моделі.

Узадачах медичної візуалізації використання VAE є доцільним, оскільки дозволяє виявляти відхилення від нормального стану без необхідності повністю розміченого датасету. Це особливо важливо у випадках, коли патологічні дані обмежені або складні для анотації.

Хід роботи

1. Завантаження та підготовка датасету

На даному етапі було виконано завантаження датасету рентгенівських зображень грудної клітки за допомогою Kaggle API у середовищі Google Colab. Використаний датасет є спрощеною альтернативою NIH Chest X ray14 і містить зображення, поділені на класи NORMAL та PNEUMONIA. Після завантаження архів було розпаковано, а структура директорій проаналізована для визначення шляхів до навчальних та тестових вибірок. Було встановлено, що навчальна вибірка містить лише зображення класу NORMAL, що відповідає умовам задачі anomaly detection. Тестова вибірка включає як нормальні, так і патологічні зображення

```
from google.colab import files
files.upload()
```

```
Выбрать файлы Файл не выбран    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving kaggle.json to kaggle.json
{'kaggle.json': b'{"username":"bob123w","key":"42da5d99e70c0404ecc4f7d77f6b9de4"}'}
```

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!pip install -q kaggle
```

```
!kaggle datasets download -d paultimothymooney/chest-xray-pneumonia
!unzip -q chest-xray-pneumonia.zip -d chest_xray
```

```
Dataset URL: https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia
License(s): other
Downloading chest-xray-pneumonia.zip to /content
100% 2.29G/2.29G [00:20<00:00, 123MB/s]
```

```
train_normal_path = chest_xray/chest_xray/chest_xray/train/NORMAL
test_normal_path = chest_xray/chest_xray/chest_xray/test/NORMAL
test_pneumonia_path = chest_xray/chest_xray/chest_xray/test/PNEUMONIA
Train NORMAL: 1341
Test NORMAL: 234
Test PNEUMONIA: 390
```

2. Попередня обробка зображень

На цьому етапі було виконано приведення зображень до єдиного формату. Зображення були змінені до однакового розміру та нормалізовані у діапазон від 0 до 1. Це є необхідним для коректної роботи нейронної мережі, оскільки дозволяє стабілізувати навчання та забезпечити однаковий масштаб вхідних даних. Також було сформовано TensorFlow датасети з використанням батчів, що дозволяє ефективно обробляти великі обсяги даних та прискорює процес навчання. Дані були перемішані, що запобігає залежності результатів від порядку подачі зображень.

```python
import tensorflow as tf

IMG_SIZE = 128

def load_image(path):
    image = tf.io.read_file(path)
    image = tf.image.decode_jpeg(image, channels=1)
    image = tf.image.resize(image, [IMG_SIZE, IMG_SIZE])
    image = tf.cast(image, tf.float32) / 255.0
    return image
```

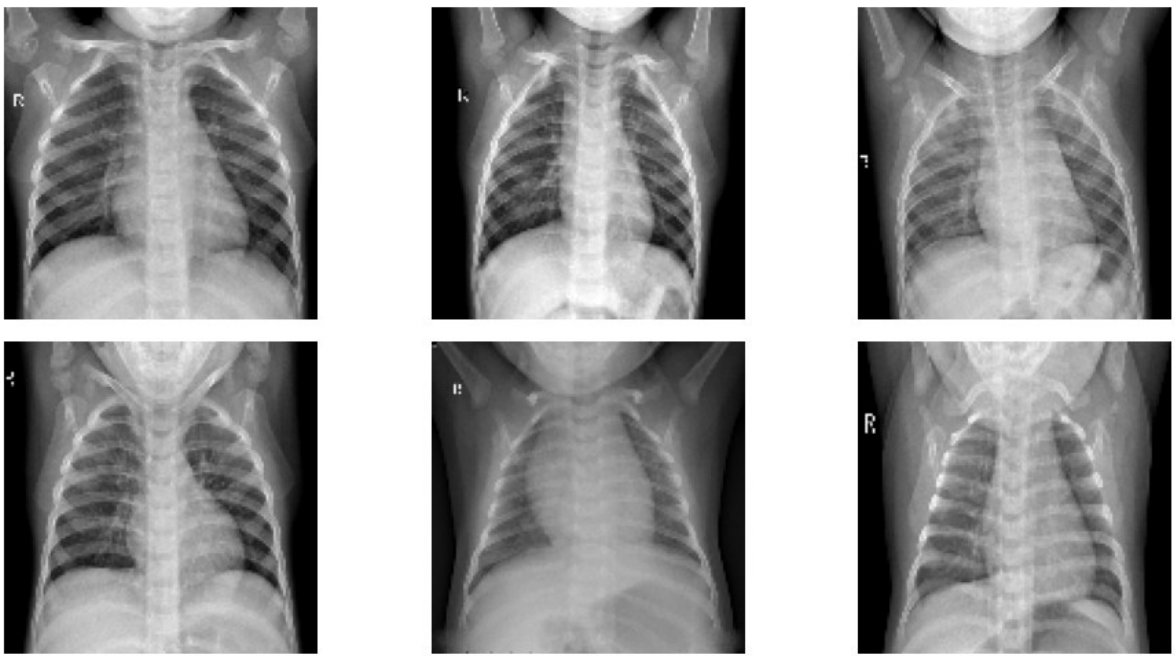
3. Візуальна перевірка зображень

Перед навчанням моделі було виконано візуальну перевірку декількох зображень із навчального датасету. Це дозволило переконатися, що зображення коректно зчитуються, мають правильний формат і не містять пошкоджених даних.

```python
import matplotlib.pyplot as plt

sample_batch = next(iter(train_ds))

plt.figure(figsize=(10, 5))
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(sample_batch[i].numpy().squeeze(), cmap="gray")
    plt.axis("off")
plt.tight_layout()
plt.show()
```



4. Побудова архітектури VAE

Було реалізовано варіаційний автоенкодер, що складається з енкодера та декодера. Енкодер виконує перетворення вхідного зображення у латентний простір, формуючи параметри розподілу, зокрема середнє значення та логарифм дисперсії. На основі цих параметрів виконується семплінг латентного вектора.

Декодер, у свою чергу, виконує зворотне перетворення, відновлюючи зображення з латентного представлення. Таким чином, модель навчається стискати інформацію та відновлювати її з мінімальними втратами.

```
from tensorflow import keras
from tensorflow.keras import layers
import tensorflow as tf

class Sampling(layers.Layer):
    def call(self, inputs):
        z_mean, z_log_var = inputs
        epsilon = tf.random.normal(shape=tf.shape(z_mean))
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon
```

Енкодер

Model: "encoder"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 128, 128, 1)	0	-
conv2d (Conv2D)	(None, 64, 64, 32)	320	input_layer[0][0]
batch_normalization (BatchNormalizatio…	(None, 64, 64, 32)	128	conv2d[0][0]
re_lu (ReLU)	(None, 64, 64, 32)	0	batch_normalizat…
conv2d_1 (Conv2D)	(None, 32, 32, 64)	18,496	re_lu[0][0]
batch_normalizatio… (BatchNormalizatio…	(None, 32, 32, 64)	256	conv2d_1[0][0]
re_lu_1 (ReLU)	(None, 32, 32, 64)	0	batch_normalizat…
conv2d_2 (Conv2D)	(None, 16, 16, 128)	73,856	re_lu_1[0][0]
batch_normalizatio… (BatchNormalizatio…	(None, 16, 16, 128)	512	conv2d_2[0][0]
re_lu_2 (ReLU)	(None, 16, 16, 128)	0	batch_normalizat…
flatten (Flatten)	(None, 32768)	0	re_lu_2[0][0]
dense (Dense)	(None, 128)	4,194,432	flatten[0][0]
batch_normalizatio… (BatchNormalizatio…	(None, 128)	512	dense[0][0]
re_lu_3 (ReLU)	(None, 128)	0	batch_normalizat…
dense_1 (Dense)	(None, 16)	2,064	re_lu_3[0][0]
dense_2 (Dense)	(None, 16)	2,064	re_lu_3[0][0]
sampling (Sampling)	(None, 16)	0	dense_1[0][0], dense_2[0][0]

Total params: 4,292,640 (16.38 MB)
Trainable params: 4,291,936 (16.37 MB)
Non-trainable params: 704 (2.75 KB)

Декодер

Model: "decoder"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 16)	0
dense_3 (Dense)	(None, 32768)	557,056
batch_normalization_4 (BatchNormalization)	(None, 32768)	131,072
re_lu_4 (ReLU)	(None, 32768)	0
reshape (Reshape)	(None, 16, 16, 128)	0
conv2d_transpose (Conv2DTranspose)	(None, 32, 32, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 32, 32, 128)	512
re_lu_5 (ReLU)	(None, 32, 32, 128)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 64, 64, 64)	73,792
batch_normalization_6 (BatchNormalization)	(None, 64, 64, 64)	256
re_lu_6 (ReLU)	(None, 64, 64, 64)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 128, 128, 32)	18,464
batch_normalization_7 (BatchNormalization)	(None, 128, 128, 32)	128
re_lu_7 (ReLU)	(None, 128, 128, 32)	0
conv2d_3 (Conv2D)	(None, 128, 128, 1)	289

Total params: 929,153 (3.54 MB)
Trainable params: 863,169 (3.29 MB)
Non-trainable params: 65,984 (257.75 KB)

5. Інтеграція Batch Normalization

У архітектуру моделі було інтегровано шари Batch Normalization після згорткових і повнозв’язних шарів. Це дозволило нормалізувати активації нейронної мережі на кожному кроці навчання. Використання BatchNorm сприяє стабілізації процесу оптимізації, зменшує внутрішнє зміщення розподілу активацій і дозволяє використовувати більші значення швидкості навчання без ризику розходження моделі.

```
def build_encoder():
    inputs = keras.Input(shape=(IMG_SIZE, IMG_SIZE, 1))

    x = layers.Conv2D(32, 3, strides=2, padding="same")(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)

    x = layers.Conv2D(64, 3, strides=2, padding="same")(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)

    x = layers.Conv2D(128, 3, strides=2, padding="same")(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)

    x = layers.Flatten()(x)
    x = layers.Dense(128)(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)

    z_mean = layers.Dense(LATENT_DIM)(x)
    z_log_var = layers.Dense(LATENT_DIM)(x)
    z = Sampling()([z_mean, z_log_var])
```

```
def build_decoder():
    inputs = keras.Input(shape=(LATENT_DIM,))

    x = layers.Dense(16 * 16 * 128)(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)

    x = layers.Reshape((16, 16, 128))(x)

    x = layers.Conv2DTranspose(128, 3, strides=2, padding="same")(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)

    x = layers.Conv2DTranspose(64, 3, strides=2, padding="same")(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)

    x = layers.Conv2DTranspose(32, 3, strides=2, padding="same")(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)

    outputs = layers.Conv2D(1, 3, activation="sigmoid", padding="same")(x)

    return keras.Model(inputs, outputs, name="decoder")

decoder = build_decoder()
decoder.summary()
```

6. Навчання моделі

Модель була навчена на нормальних зображеннях протягом 10 епох. У процесі навчання відслідковувалися значення загальної функції втрат, reconstruction loss та KL loss.

```
vae = VAE(encoder, decoder)
vae.compile(optimizer=keras.optimizers.Adam())

history = vae.fit(train_ds, epochs=10)
```

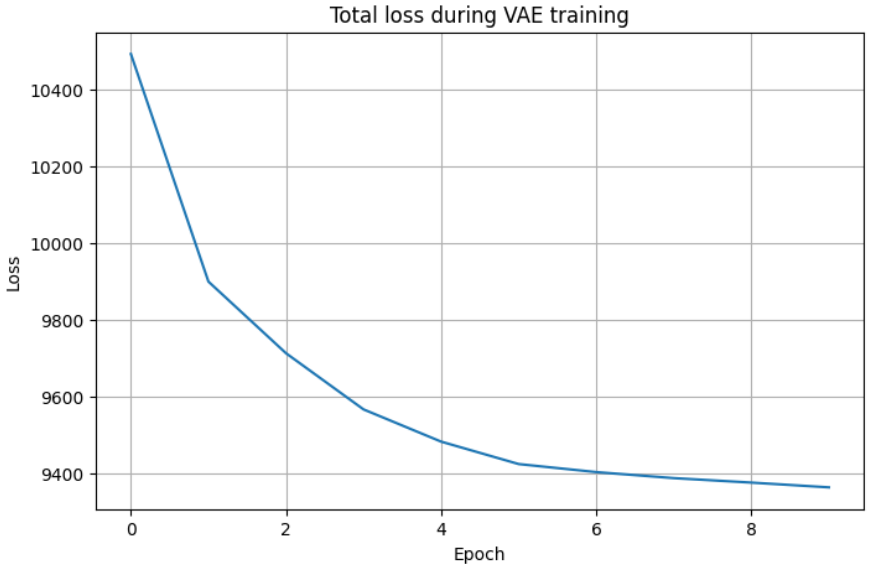
```
Epoch 1/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 39s 285ms/step - kl_loss: 15.9676 - loss: 10492.6543 - reconstruction_loss: 10476.6885
Epoch 2/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 16s 99ms/step - kl_loss: 14.0854 - loss: 9899.7314 - reconstruction_loss: 9885.6455
Epoch 3/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 16s 117ms/step - kl_loss: 25.6556 - loss: 9713.1582 - reconstruction_loss: 9687.4990
Epoch 4/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 20s 98ms/step - kl_loss: 28.9863 - loss: 9566.7734 - reconstruction_loss: 9537.7861
Epoch 5/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 15s 98ms/step - kl_loss: 30.9386 - loss: 9482.8604 - reconstruction_loss: 9451.9219
Epoch 6/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 16s 111ms/step - kl_loss: 32.3316 - loss: 9424.4336 - reconstruction_loss: 9392.1035
Epoch 7/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 16s 131ms/step - kl_loss: 32.1042 - loss: 9403.4736 - reconstruction_loss: 9371.3691
Epoch 8/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 20s 98ms/step - kl_loss: 32.3903 - loss: 9387.7871 - reconstruction_loss: 9355.3955
Epoch 9/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 20s 99ms/step - kl_loss: 32.3307 - loss: 9376.1484 - reconstruction_loss: 9343.8174
Epoch 10/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 15s 97ms/step - kl_loss: 33.0738 - loss: 9363.7510 - reconstruction_loss: 9330.6768
```

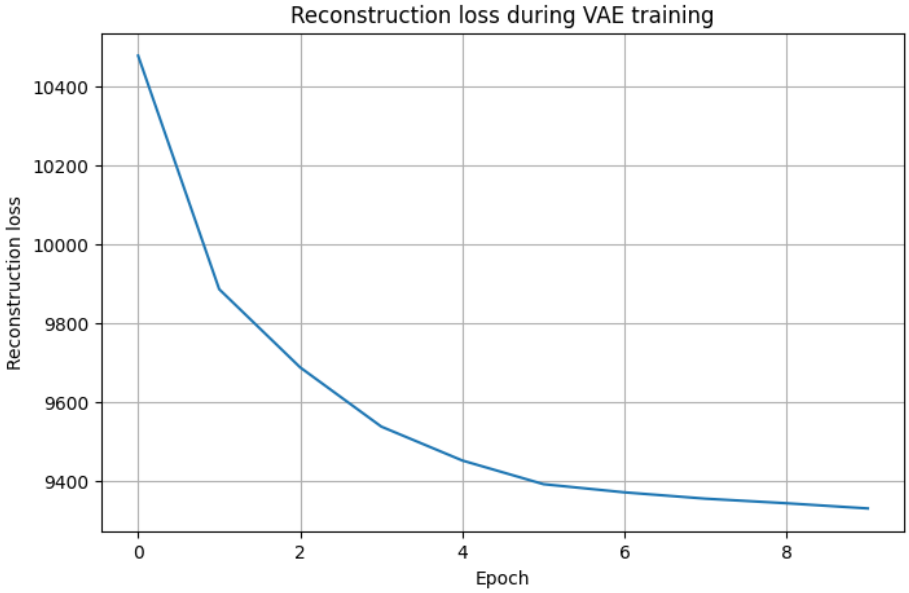
Аналіз результатів показав, що значення функції втрат поступово зменшується з кожною епохою. Наприклад, загальний loss зменшився приблизно з 10492 до 9363, що свідчить про покращення якості реконструкції. Reconstruction loss також зменшився, що означає, що модель краще відновлює вхідні зображення.

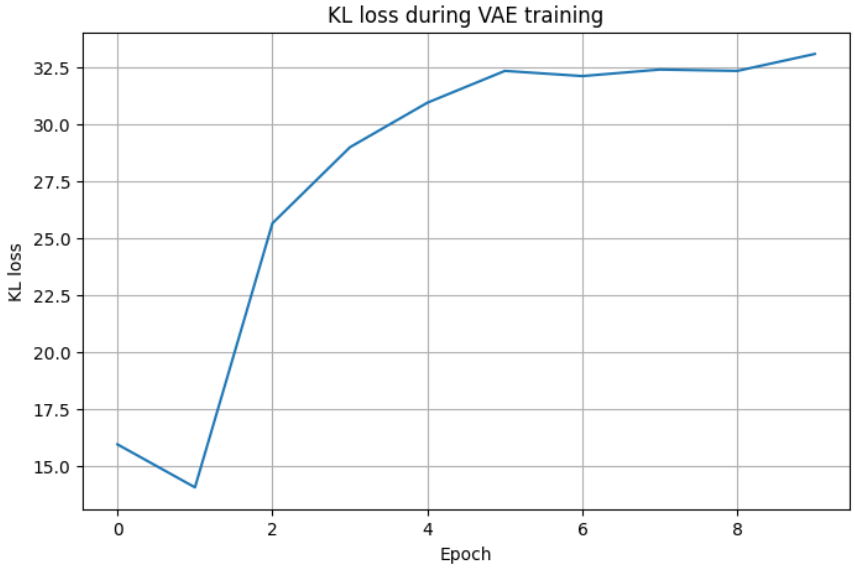
Значення KL loss зростає, що є нормальним явищем для VAE, оскільки модель поступово формує більш структурований латентний простір.

7. Аналіз стабільності навчання

Було побудовано графіки зміни функції втрат у процесі навчання.







Аналіз графіків показав плавне зменшення значень без різких стрибків або коливань. Це свідчить про стабільність навчання моделі. Така поведінка є результатом використання Batch Normalization, яка нормалізує активації та запобігає нестабільності градієнтів.

8. Реконструкція зображень

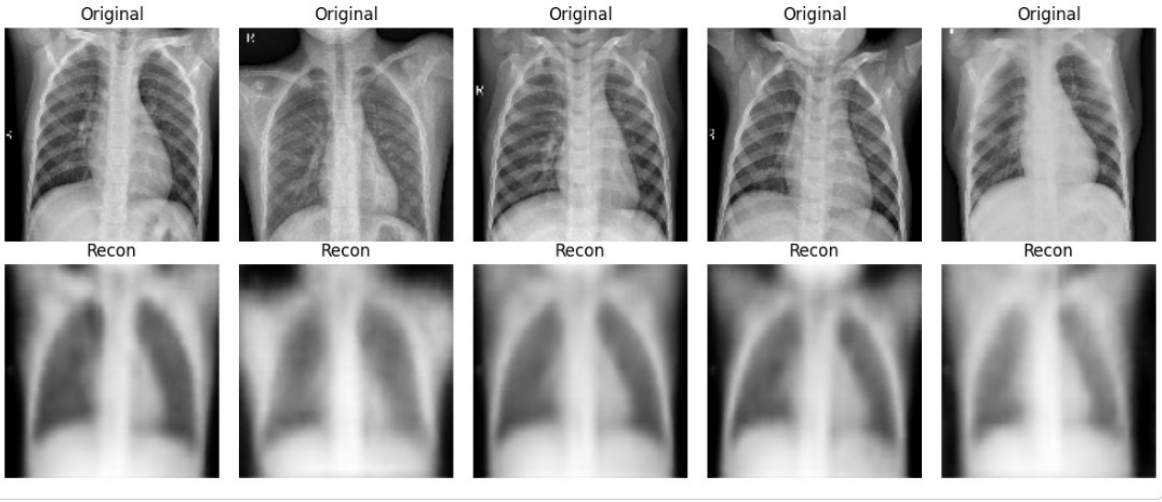
Було виконано реконструкцію зображень для класів NORMAL та PNEUMONIA.

Аналіз показав, що модель добре відновлює загальну структуру нормальних зображень, хоча реконструкції є дещо розмитими. Це є типовою властивістю VAE, який стискає інформацію у латентному просторі.

Для патологічних зображень реконструкція є значно гіршою, втрачаються деталі та характерні ознаки патології. Це пояснюється тим, що модель не навчалася на таких даних і не може їх коректно відновити.

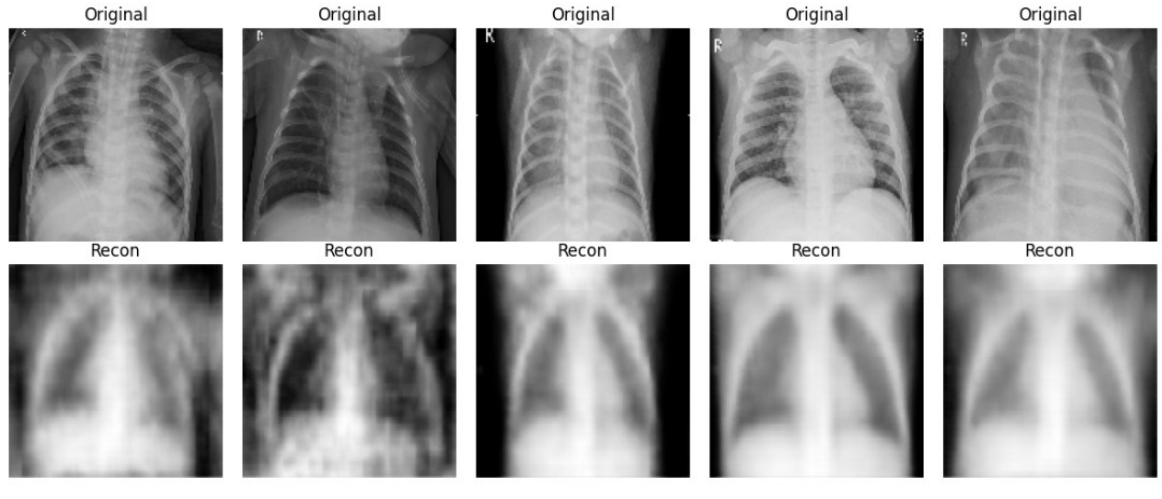
Normal

```
show_reconstructions(vae, test_normal_ds, n=5)
```



Pneumonia

```
show_reconstructions(vae, test_pneumonia_ds, n=5)
```



9. Обчислення reconstruction error

Було обчислено середню помилку реконструкції для обох класів.

```
Normal mean error: 0.010032875
Pneumonia mean error: 0.01175723
```

Аналіз показує, що помилка для патологічних зображень є вищою. Це підтверджує, що модель гірше відновлює аномальні дані, що і використовується для їх виявлення.

10. Визначення порогу та класифікація

9. Визначення порогу (threshold)

визначається порогове значення reconstruction error, яке використовується для класифікації зображень як нормальних або аномальних. Поріг обчислюється на основі статистичних характеристик помилки реконструкції для нормальних зображень, зокрема як сума середнього значення та стандартного відхилення.

```
threshold = normal_errors.mean() + normal_errors.std()

print("Threshold:", threshold)
```

```
Threshold: 0.013148577
```

10. Виявлення аномалій

```
normal_anomalies = np.sum(normal_errors > threshold)
pneumonia_anomalies = np.sum(pneumonia_errors > threshold)

print("NORMAL classified as anomaly:", normal_anomalies, "out of", len(normal_errors))
print("PNEUMONIA classified as anomaly:", pneumonia_anomalies, "out of", len(pneumonia_errors))
```

```
NORMAL classified as anomaly: 34 out of 234
PNEUMONIA classified as anomaly: 95 out of 390
```

11. Аналіз латентного простору

Було виконано зменшення розмірності латентного простору за допомогою PCA та побудовано графік розподілу.



Аналіз показав, що між класами існують відмінності у латентному просторі, хоча повного розділення не спостерігається. Це є очікуваним результатом, оскільки модель не була навчена явно розрізняти класи.

Висновок

У ході виконання лабораторної роботи було реалізовано варіаційний автоенкодер (VAE) для задачі виявлення аномалій у медичних зображеннях. Модель була навчена виключно на нормальних рентгенівських зображеннях, що дозволило сформувати латентний простір, характерний для здорових даних.

У процесі експерименту було встановлено, що модель здатна коректно відновлювати нормальні зображення, зберігаючи їх основну структуру, тоді як для патологічних зображень реконструкція є менш точною та більш розмитою. Кількісний аналіз показав, що значення reconstruction error для зображень класу PNEUMONIA є вищим, ніж для класу NORMAL, що підтверджує можливість використання цього показника для виявлення аномалій.

Було також проведено аналіз латентного простору, який показав наявність відмінностей у представленні нормальних і патологічних зображень. Незважаючи на часткове перекриття класів, латентний простір містить інформацію, корисну для задачі anomaly detection.

В рамках індивідуального завдання було інтегровано Batch Normalization у архітектуру VAE. Це дозволило стабілізувати процес навчання, що підтверджується плавною динамікою функції втрат та відсутністю різких коливань під час оптимізації. Таким чином, використання BatchNorm сприяло більш стабільній та ефективній збіжності моделі.

Посилання на Google Colab :

https://colab.research.google.com/drive/16Uns4VnK1HfmNLDdCsNykBFQB2ysdYN2?usp=sharing